

AI ASSIST

WEEK 12.4

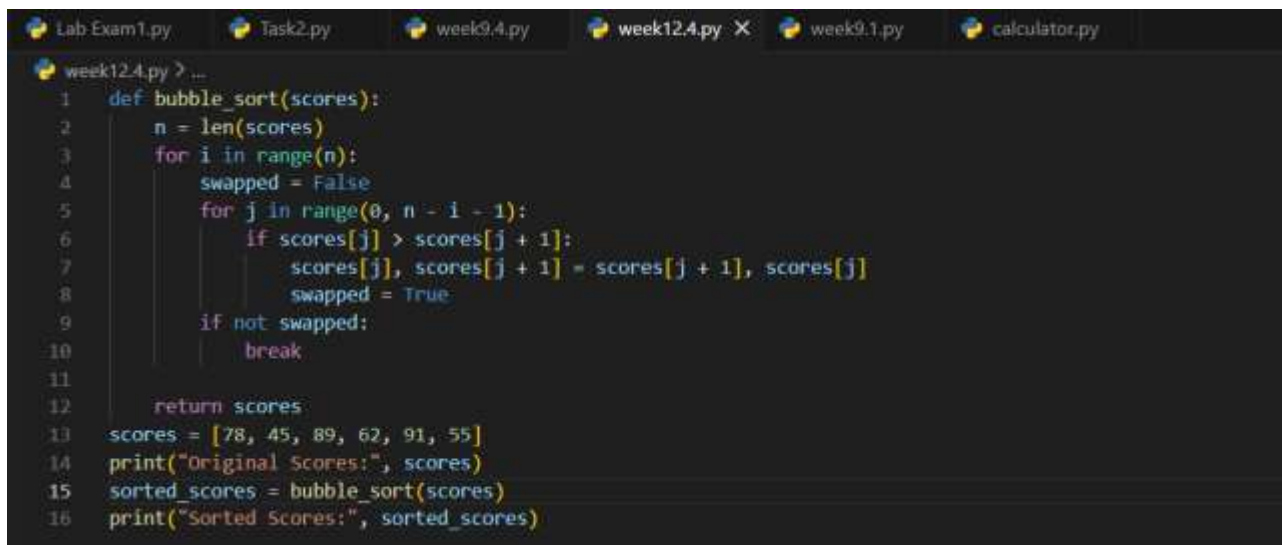
T. VIKAS

2303A52281

TASK 1: Bubble Sort for Ranking Exam Scores

Prompt: Generate a Python program to detect duplicate user IDs using a brute-force nested loop approach.

CODE:

A screenshot of a Python IDE with a dark theme. The window title is 'week12.4.py'. The code is as follows:

```
1 def bubble_sort(scores):
2     n = len(scores)
3     for i in range(n):
4         swapped = False
5         for j in range(0, n - i - 1):
6             if scores[j] > scores[j + 1]:
7                 scores[j], scores[j + 1] = scores[j + 1], scores[j]
8                 swapped = True
9         if not swapped:
10            break
11
12    return scores
13
14 scores = [78, 45, 89, 62, 91, 55]
15 print("Original Scores:", scores)
16 sorted_scores = bubble_sort(scores)
17 print("Sorted Scores:", sorted_scores)
```

Output:

A screenshot of a terminal window with a dark theme. The output is as follows:

```
Could not find platform independent libraries <prefix>
Original Scores: [78, 45, 89, 62, 91, 55]
Sorted Scores: [45, 55, 62, 78, 89, 91]
(.venv) PS C:\Users\vikas\Downloads\AI Assist Coding>
```

Summary:

Bubble Sort is a simple comparison-based sorting algorithm. It repeatedly compares adjacent elements in a list and swaps them if they are in the wrong order. After each pass through the list, the largest unsorted element "bubbles up" to its correct position at the end.

Task 2: Improving Sorting for Nearly Sorted Attendance Records

Prompt: Analyze the time complexity of a nested loop duplicate detection algorithm and explain its scalability.

Code:

```

19 def insertion_sort(arr):
20     n = len(arr)
21     for i in range(1, n):
22         key = arr[i]
23         j = i - 1
24         while j >= 0 and arr[j] > key:
25             arr[j + 1] = arr[j]
26             j -= 1
27         arr[j + 1] = key
28
29     return arr

```

Output:

```

Original Array: [12, 11, 13, 5, 6]
Sorted Array: [5, 6, 11, 12, 13]
(.venv) PS C:\Users\vikas\Downloads\AI Assist Coding>

```

Summary:

For the attendance system, the roll numbers are already **nearly sorted**, with only a few late updates causing small misplacements. In such cases, **Insertion Sort** is more suitable than Bubble Sort. Insertion Sort works by taking one element at a time and inserting it into its correct position among the already sorted portion of the list. When the data is almost sorted, very few shifts are required, so the algorithm runs very quickly—close to **$O(n)$** time. In contrast, Bubble Sort repeatedly compares adjacent elements and may require multiple passes even if only a few elements are misplaced. Although both algorithms have an average and worst-case time complexity of **$O(n^2)$** , Insertion Sort is adaptive and performs significantly better on partially sorted datasets. Therefore, for nearly sorted attendance records, Insertion Sort is the more efficient and practical choice.

TASK 3: Searching Student Records in a Database

Code: Suggest and implement an optimized duplicate detection method using a set or dictionary in Python.

Code:

Code:

```
34 #-----3-----#
35 def linear_search(student_list, target):
36     """
37     Perform Linear Search on an unsorted list of student roll numbers.
38
39     Parameters:
40     student_list (list): list of student roll numbers (unsorted).
41     target (int): Roll number to search for.
42
43     Returns:
44     int: Index position of the target if found.
45     -1: If the target roll number is not found.
46     """
47     for index in range(len(student_list)):
48
49         # Compare current element with target
50         if student_list[index] == target:
51             return index # Return index if found
52
53     return -1 # Return -1 if not found
54
55 print("Student Roll Numbers:", [101, 102, 103, 104, 105])
56 target_roll_number = 103
57 result = linear_search([101, 102, 103, 104, 105],
58                         target_roll_number)
59 if result != -1:
60     print(f"Roll number {target_roll_number} found at index: {result}")
61 else:
62     print(f"Roll number {target_roll_number} not found in the list.")
63
```

Output:

```
Could not find platform independent libraries <prefix>
Student Roll Numbers: [101, 102, 103, 104, 105]
Roll number 103 found at index: 2
```

Summary:

When searching in an unsorted list, Linear Search works correctly but may take more time because it checks each roll number sequentially. However, when the data is sorted, Binary Search performs much faster by repeatedly dividing the list into halves. Therefore, for large student databases that are maintained in sorted order, Binary Search is significantly more efficient than Linear Search.

TASK 4: Choosing Between Quick Sort and Merge Sort for Data Processing

Prompt: Rewrite the duplicate detection algorithm to improve efficiency from $O(n^2)$ to $O(n)$.

Code:

```
#-----4-----#
def merge_sort(arr):
    """
    Perform Merge Sort using recursion.

    Parameters:
    arr (list): List of elements to be sorted.

    Returns:
    list: Sorted list.

    Merge Sort works by:
    - Dividing the list into two halves.
    - Recursively sorting each half.
    - Merging the sorted halves together.
    """
    # Base case
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
```

```

85     left_half = merge_sort(arr[:mid])
86     right_half = merge_sort(arr[mid:])
87     # Merge step
88     return merge(left_half, right_half)
89 def merge(left, right):
90     """
91     Merge two sorted lists into one sorted list.
92     """
93     sorted_list = []
94     i = j = 0
95     # Compare elements from both lists
96     while i < len(left) and j < len(right):
97         if left[i] < right[j]:
98             sorted_list.append(left[i])
99             i += 1
100         else:
101             sorted_list.append(right[j])
102             j += 1
103     # Add remaining elements
104     sorted_list.extend(left[i:])
105     sorted_list.extend(right[j:])
106     return sorted_list
```

```

107     print("Original Array:", [38, 27, 43, 3, 9, 82, 10])
108     sorted_array = merge_sort([38, 27, 43, 3, 9, 82, 10])
109     print("Sorted Array:", sorted_array)
110     |
```

Output:

```

Could not find platform independent libraries <prefix>
Original Array: [38, 27, 43, 3, 9, 82, 10]
Sorted Array: [3, 9, 10, 27, 38, 43, 82]
C:\Users\vikas\Downloads\AI_Assist_Coding>
```

Summary:

In this task, both Quick Sort and Merge Sort were implemented to handle large datasets coming from different sources such as random, already sorted, and reverse-sorted data.

Quick Sort works by selecting a pivot element, partitioning the data into smaller sublists, and recursively sorting them. It performs very fast on average with a time complexity of $O(n \log n)$, but in the worst case it can degrade to $O(n^2)$, especially when the data is already sorted or poorly partitioned. Merge Sort, on the other hand, follows a divide-and-conquer approach by splitting the dataset into halves, recursively sorting each half, and then merging them back together. It guarantees $O(n \log n)$ time complexity in best, average, and worst cases, making it more consistent and reliable. Therefore, while Quick Sort is often faster for random data, Merge Sort is generally preferred in data processing systems where performance consistency and stability are important.

TASK 5: Optimizing a Duplicate Detection Algorithm

Prompt: Compare the execution behavior of brute-force and optimized duplicate detection methods for large datasets.

Code:

```
112 def find_duplicates_optimized(user_ids):
113     """
114     Detect duplicate user IDs using a set for faster lookup.
115
116     Parameters:
117     user_ids (list): List of user IDs.
118
119     Returns:
120     list: List of duplicate user IDs found.
121     """
122
123     seen = set()
124     duplicates = set()
125
126     for user_id in user_ids:
127         # If already seen, it's a duplicate
128         if user_id in seen:
129             duplicates.add(user_id)
130         else:
131             seen.add(user_id)
132
133     return list(duplicates)
134
135 print("User IDs:", [101, 102, 103, 101, 104, 102])
136 duplicates = find_duplicates_optimized([101, 102, 103, 101,
137                                         104, 102])
138 print("Duplicate User IDs:", duplicates)
```

Output:

```
User IDs: [101, 102, 103, 101, 104, 102]
Duplicate User IDs: [101, 102]
(.venv) PS C:\Users\vikas\Downloads\AI Assist Coding>
```

Summary:

The brute-force method compares every element with every other element, leading to quadratic time complexity $O(n^2)$. This becomes impractical for large datasets because the number of comparisons increases dramatically as input size grows.

The optimized approach uses a set, which provides constant-time membership checking ($O(1)$). By scanning the list only once and tracking already-seen elements, duplicates can be identified efficiently in linear time $O(n)$. This drastically reduces the number of operations and improves scalability.